pve-fcsan Runbook

Version 1.6.0

Purpose and scope

`pvesan` (Debian package `pve-fcsan`) gives Proxmox VE the SAN storage workflow vCenter has: list HBAs, rescan, manage LUNs and paths, and create, grow, rename, mount, unmount, reclaim and delete shared datastores on Fibre Channel LUNs. With an array driver it also creates and maps the volume on the array. You can use it from the web GUI (**Node → Disks → SAN / Fibre Channel**), from the `pvesan` CLI, or through the REST API under `/nodes/{node}/san`.

A pvesan *datastore* is a shared thick LVM volume group on one or more multipathed LUNs. It is registered in `/etc/pve/storage.cfg` as storage type `fcsan`, a subclass of Proxmox's built-in LVM plugin, so live migration and HA work the same way they do on shared LVM. iSCSI LUNs also show up in the device lists and work the same way, but FC is the main target.

vCenter action	pvesan GUI	pvesan CLI
Rescan Storage	Adapters → Rescan / Rescan all nodes	`pvesan rescan` / `pvesan rescan-all`
Storage Adapters view	Adapters tab	`pvesan adapters`
Storage Devices view	Devices tab	`pvesan devices`
Datastore capacity, performance, related VMs	Datastores → Array view / VMs; Performance tab	`pvesan datastore array-view`, `pvesan iostats`, `pvesan datastore guests`
Storage alarms, path health	Health tab; Proxmox notifications	`pvesan path-health`
Paths view, path state	Paths tab	`pvesan paths`
Edit Multipathing (Fixed / MRU / Round Robin)	Devices → Properties → Path policy	`pvesan device set <wwid> --policy ...`
Device display name	Devices → Properties → Alias	`pvesan device set <wwid> --alias ...`
New Datastore (VMFS)	Datastores → Create	`pvesan datastore create`
Increase Datastore Capacity (grow LUN)	Datastores → Increase capacity	`pvesan datastore extend <id>`
Increase Capacity (add extent)	Datastores → Increase capacity → Add LUN	`pvesan datastore extend <id> --add-devices ...`
Rename datastore	Datastores → Rename	`pvesan datastore rename <id> --label ...` (or `--new-id`)
Unmount / Mount datastore	Datastores → Unmount / Mount	`pvesan datastore unmount` / `mount`
Delete datastore	Datastores → Delete	`pvesan datastore delete <id>`
Detach / Attach device	Devices → Detach / Attach	`pvesan device detach` / `attach`
Reclaim space (UNMAP)	Datastores → Reclaim	`pvesan datastore reclaim <id>`

vCenter action	pvesan GUI	pvesan CLI
Remove dead paths after unmap	Devices → Clean up dead paths	`pvesan device cleanup`
Array plugin: new datastore on a new volume	Arrays → Provision datastore	`pvesan provision <array> <id> ...`
Array plugin: snapshot, restore VM from snapshot	Snapshots tab; Datastores → Restore VM	`pvesan snapshot ...`, `pvesan copy restore-vm`

Not in scope: you can't shrink a datastore (vCenter can't either) and there is no Storage I/O Control. Array snapshots, mounting them as copies and restoring VMs from them (P13 to P15) need a driver with snapshot support; for other arrays the troubleshooting section gives a manual resignature procedure.

Prerequisites

Every cluster node needs the same LUNs, presented the same way, before pvesan can build a datastore. pvesan checks this and refuses to create a datastore on a LUN that some node can't see.

Software

- Proxmox VE 8.x or 9.x on every node, all on the same pve-fcsan version.
- `multipath-tools` and `lvm2`. The package pulls them in.
- `multipathd` enabled: `systemctl enable --now multipathd`.

Fabric and array

1. Zone each node's HBA ports (WWPNs) to the array target ports. Single-initiator zoning is best practice. Get the WWPNs with `pvesan adapters` or from the GUI **Adapters** tab.
2. Put every node's WWPNs in one host group / host cluster on the array, with the same LUN ID on every node. With an array driver, pvesan does this for you (P12).
3. Turn on ALUA on the array if it supports it.
4. Use thin LUNs with UNMAP turned on if you want space reclaim.

Multipath base configuration

pvesan writes only `/etc/multipath/conf.d/pvesan.conf` and `/etc/multipath/wwids`. Keep `/etc/multipath.conf` identical on every node. A good starting point:

```
defaults {
    find_multipaths     strict
    user_friendly_names no
    polling_interval    5
    path_selector       "service-time 0"
}
```

Add the `devices { }` settings from your array vendor's Linux host guide. Don't list pvesan-managed WWIDs in a `multipaths { }` section there, because two sections for the same WWID conflict. `user_friendly_names no` matters: map names must be the same on every node.

LVM

- Leave `devices/multipath_component_detection = 1` as it is (the default). It stops LVM from using the raw `sdX` paths.
- If `use_devicesfile = 1` is set, pvesan adds each datastore's `/dev/mapper` device to the devices file on every node for you.
- Optional: set `devices/issue_discards = 1` in `/etc/lvm/lvm.conf` on every node. Space from deleted VM disks then goes back to a thin array right away, like VMFS6 automatic UNMAP.

Permissions

Action	Required privilege
View adapters, devices, paths, datastores	`Sys.Audit` on `/nodes/{node}`
Rescan, clean up paths on one node	`Sys.Modify` on `/nodes/{node}`
Any cluster-wide action (datastores, aliases, policies, detach, rescan all, arrays)	`Sys.Modify` on `/` and `Datastore.Allocate` on `/storage`

Installation, upgrade, uninstall

Install the same `pve-fcsan` .deb on every node. A node without it can't read `fcsan` entries in `storage.cfg`.

From the apt repository (recommended). On each node, as root:

```
curl -fsSL https://warddynamiccomputing.github.io/pve-fcsan/pve-fcsan.gpg -o /usr/share/keyrings/pve-fcsan.gpg
cat > /etc/apt/sources.list.d/pve-fcsan.sources <<'EOF'
Types: deb
URIs: https://warddynamiccomputing.github.io/pve-fcsan
Suites: stable
Components: main
Signed-By: /usr/share/keyrings/pve-fcsan.gpg
EOF
apt update
apt install pve-fcsan
```

After that, `apt upgrade` installs new pve-fcsan versions along with Proxmox VE updates. The repository's signing key has the fingerprint `FBCA 7A62 0928 1FAA 796F AFD0 ACCA EEE8 1834 4A99`; check it with `gpg --show-keys /usr/share/keyrings/pve-fcsan.gpg`. Then continue with the status check below.

Or get the package yourself. Download it from the project's GitHub releases, or build it on any Debian 12/13 machine or Proxmox node:

```
apt install build-essential debhelper git
git clone <repository-url> pve-fcsan
cd pve-fcsan
make test
make deb
```

Install on each node, one at a time:

```
apt install ./pve-fcsan_1.6.0_all.deb
/usr/share/pve-fcsan/pvesan-integrate status
pvesan adapters
```

The status command should report both `Nodes.pm` and `index.html.tpl` as `integrated`. Reload the browser with Ctrl+F5, then open **Node → Disks → SAN / Fibre Channel**.

Installing does four things:

1. Installs the storage plugin to `/usr/share/perl5/PVE/Storage/Custom/FCSANPlugin.pm`.
2. Adds one marked line each to `PVE/API2/Nodes.pm` (API) and `index.html.tpl` (GUI). If `Nodes.pm` doesn't compile with the line added, the change is reverted. The API line loads pve-fcsan inside an `eval`: if pve-fcsan ever fails to load, only the SAN API and panel go away, and the rest of the Proxmox VE API keeps working.
3. Enables `pvesan-sync.service`. At boot it applies the cluster-wide aliases, policies and detached-LUN list before guests start. It also enables `pvesan-monitor.timer` (health checks and alerts, see Monitoring and health) and installs the `pvesan-alert` notification templates in `/usr/share/pve-manager/templates/default/`.
4. Reloads `pvedaemon`, `pveproxy` and `pvestatd`.

Upgrading Proxmox VE. A pve-manager upgrade replaces `Nodes.pm` and `index.html.tpl`. A dpkg trigger re-runs the integration and reloads the services on its own. After any Proxmox VE upgrade, check:

```
/usr/share/pve-fcsan/pvesan-integrate status
```

If it says `integrated by an older pve-fcsan`, run `/usr/share/pve-fcsan/pvesan-integrate apply`. If it says `NOT integrated`, run `/usr/share/pve-fcsan/pvesan-integrate apply` and read the reason it prints. If it says `PVE::API2::SAN: FAILS to load`, the error above it names the cause; `journalctl -u pveproxy` shows the same warning. The storage plugin and existing datastores keep working without the integration; only the GUI panel and the `/san` API go away until it is fixed.

Uninstall. Package removal is blocked while `fcsan` storages exist, because Proxmox VE would otherwise drop them and guests would fail to start. Convert each one to plain LVM first; the data and the VG stay as they are.

1. Edit `/etc/pve/storage.cfg` on any node. Change `fcsan: <id>` to `lvm: <id>` and delete the `fc-wwids` and `fc-autorescan` lines.
2. Run `apt remove pve-fcsan` on every node. Use `apt purge` to also remove `/etc/multipath/conf.d/pvesan.conf`.
3. If you still want the aliases after a purge, copy them into `/etc/multipath.conf`.

Procedures

All cluster-wide actions run as a task on the node you issue them from, which then drives every online node. The task log, in the GUI task list or on CLI stdout, prints a `[node]` line for each step. You can run every procedure from any node.

P1. Rescan storage

Use this after you present, unpresent or grow LUNs on the array.

- **GUI:** SAN → Adapters → **Rescan all nodes**.
- **CLI:** `pvesan rescan-all`, or `pvesan rescan` for this node only.

A rescan scans every FC and iSCSI host for new targets and LUNs and re-reads the capacity of existing LUNs. It also resizes any multipath map whose LUN grew, and removes paths of detached LUNs again.

Use **Rescan with LIP** (`--lip 1`) only when the fabric doesn't show new targets. A LIP logs each port in to the fabric again and pauses I/O on that port for a moment. pvesan does one port at a time, so multipath keeps I/O running on the other ports.

P2. Add a new LUN

1. On the array, map the LUN to the host group that holds every node.
2. Run P1 (rescan all nodes).
3. Go to SAN → Devices. The LUN shows with usage `free` and the flag `not multipathed`.
4. Optional: select it and click **Add (multipath)**, or run `pvesan device add <wwid> --alias <name>`. Creating a datastore does this for you.

P3. Alias and path policy

Select the LUN, then **Properties**, or run:

```
pvesan device set <wwid> --alias ds-gold-01 --policy alua
```

Policy	vSphere equivalent	Use for
`alua`	RR on an ALUA array	Most modern arrays
`round-robin`	VMW_PSP_RR	Active/active arrays without ALUA
`mru`	VMW_PSP_MRU	Active/passive arrays that must not fail back
`fixed`	VMW_PSP_FIXED	One preferred path, fail back when it returns
`array-default`	SATP default	Settings from multipath-tools' built-in array table

Changes apply on every node at once with `multipathd reconfigure`, with no I/O outage. Changing an alias renames the `/dev/mapper` device of a LUN in use; LVM tracks devices by number, so running guests aren't affected.

P4. Create a datastore

- **GUI:** SAN → Datastores → **Create**. Enter an ID, an optional display name, content and node restriction, then tick one or more unused LUNs.
- **CLI:**

```
pvesan datastore create ds1 --devices 3600a0980383030534 53f463045727a41 --label "Gold tier 01"
```

pvesan checks that every target node sees each LUN with at least one active path (and warns about single-path LUNs), then that the LUN is free on the local node. Existing partition tables or filesystem signatures need `--force 1` (**Wipe LUNs** in the GUI). LUNs that hold a volume group are always refused.

It then registers an alias for each LUN (`ds1`, or `ds1-00`, `ds1-01` for several), applies it on all nodes, runs `pvcreate` and `vgcreate san_ds1`, and registers storage `ds1` of type `fcsan`, shared, with content `images,rootdir`. If any step fails, the LVM changes are rolled back. Check the result with `pvesm status`.

P5. Increase capacity

Grow a LUN: grow it on the array, then run `pvesan datastore extend ds1` (GUI: **Increase capacity** → **Expand**). On every node, pvesan re-reads the LUN size on each path and resizes the multipath map. It aborts if the nodes report different sizes, then runs `pvresize` under the cluster storage lock.

Add an extent:

```
pvesan datastore extend ds1 --add-devices <wwid>
```

The new LUN goes through the same checks as in P4 and is added with `vgextend`. As with VMFS extents, if any extent LUN fails, the whole datastore fails.

For array-linked datastores, see P12: one step grows the volume on the array and the datastore.

P6. Rename a datastore

- **Display name** (online, cosmetic): `pvesan datastore rename ds1 --label "Gold tier"`.
- **Storage ID** (maintenance window): `pvesan datastore rename ds1 --new-id gold01 [--rename-vg 1]`.

Before an ID rename, stop every guest whose config refers to `ds1:` and make sure every node is online. pvesan deactivates the VG on all nodes, renames the `storage.cfg` entry, optionally renames the VG to `san_gold01`, and rewrites all guest configs, snapshots included. Update backup jobs or scripts that use the old ID by hand.

P7. Unmount and mount

- **Unmount from some nodes:** `pvesan datastore unmount ds1 --nodes <node>` removes the nodes from the storage's `nodes` list and deactivates the VG there.
- **Unmount everywhere:** `pvesan datastore unmount ds1` disables the storage and deactivates the VG on every node.
- **Mount:** `pvesan datastore mount ds1 [--nodes <node>]` re-enables the storage or adds the nodes back, then activates it on each node.

Unmount is refused while guests that use the datastore are running on the target nodes.

P8. Reclaim space (UNMAP)

The Datastores **UNMAP** column shows whether every backing LUN supports discard. To reclaim:

```
pvesan datastore reclaim ds1 --reserve 100
```

pvesan allocates the free space, except the reserve in GiB, as a temporary LV, discards it, and removes it. New disks can only use the reserve while this runs, so run it outside busy periods. If a reclaim is ever interrupted, remove the leftover LV with `lvremove san_ds1/pvesan_reclaim`.

P9. Delete a datastore and retire its LUNs

1. Move or delete every guest disk on it. pvesan refuses while any guest config refers to it.
2. In the GUI, click **Delete** and type the ID, or run `pvesan datastore delete ds1 --detach 1`. Add `--destroy-data 1` to remove volumes no guest refers to.
3. pvesan deactivates the VG on all nodes, removes the storage, runs `vgremove`, `pvremove` and `wipefs`, and with `--detach` detaches the LUNs.
4. Unmap the LUNs on the array (or use P12's delete-with-volume).
5. Run P1, then `pvesan device cleanup` on each node to drop any leftover dead paths.

P10. Detach and attach a LUN

Detach is only allowed for unused LUNs. `pvesan device detach <wwid>` blacklists the WWID on every node, flushes the map and deletes every SCSI path; later rescans remove the paths again until you run `pvesan device attach <wwid>`. Always detach before you unmap a LUN on the array: unmapping an attached LUN leaves dead paths, and with `queue_if_no_path` it can hang I/O.

P11. Node maintenance

After a node reboots, `pvesan-sync.service` applies the cluster SAN config before `pvestatd` and the guests start. If a datastore's LUN is missing when Proxmox VE activates it, the `fcsan` plugin rescans the SAN on that node by itself, at most once every 5 minutes per datastore. To turn that off: `pvesm set ds1 --fc-autorescan 0`.

P12. Array integration (provisioning)

With an array driver, pvesan drives the array itself, the way the Pure Storage and INFINIDAT vSphere plugins do: one task creates the volume, maps it to the cluster, rescans and builds the datastore.

One-time setup

1. On the array, create a dedicated API user with the role listed for your array below.
2. Zone every node's FC ports to the array. **SAN** → **Adapters** lists the WWPNs. For an iSCSI array, follow P19 instead of zoning.
3. In **SAN** → **Arrays** → **Add**, choose the type, enter an ID, the management address and the credentials, plus any extra settings the type needs.

4. Click **Fetch** and compare the SHA-256 certificate fingerprint with the one in the array's management GUI. Confirm it only if it matches. pvesan pins this fingerprint and refuses any other certificate. The connection and login are tested before anything is saved, and the secret is stored root-only in `/etc/pve/priv/pvesan/`.
5. Select the array, open **Registration**, and click **Register cluster on array**. pvesan creates one host per node with its WWPNs (or reuses a host that already owns them) and puts them in a host cluster named after the Proxmox cluster, unless you set another name. Every node should then show **Registered**.

Provision a datastore

Open **SAN** → **Arrays** → **Provision datastore** (or **Datastores** → **Provision from array**). Pick the array, pool, size, thin or thick, and a datastore ID. One task then:

1. Checks the cluster registration.
2. Creates the volume.
3. Maps it to the cluster's host cluster.
4. Rescans all nodes until every node sees the LUN (timeout 120 s).
5. Sets the path policy (the vendor's recommendation unless you choose one) and creates the datastore.

If step 3, 4 or 5 fails, the task unmaps and deletes the new volume again.

CLI equivalent:

```
pvesan array probe 192.0.2.10 --type ibm-sv
pvesan array add fs01 --type ibm-sv --address 192.0.2.10 --username pvesan --fingerprint 'sha256$...'
pvesan array register-cluster fs01
pvesan provision fs01 gold01 --pool Pool0 --size 2048 --thin 1 --label "Gold 01"
```

`pvesan array add` and `array set` prompt for the password, so it never appears on the command line or in shell history.

Grow, delete, link

- **Grow:** for an array datastore, **Datastores** → **Increase capacity** asks for the new size in GiB. pvesan expands the volume on the array, every node re-reads the LUN, and the datastore grows (`pvesan datastore array-grow gold01 --size 4096`).
- **Delete:** the delete dialog has **Also unmap and delete the volume on the array**. Without it, the datastore is removed and the LUN is detached, but the volume stays on the array.
- **Link:** a datastore created by hand can be linked to its array with **Arrays** → **Link datastore**; pvesan finds its volumes by UID, and grow and delete then work as above.

Supported arrays

Array	`--type`	Port	Credentials	Extra settings	Cluster object pvesan creates	Pool	Notes
IBM FlashSystem / Spectrum Virtualize	`ibm-sv`	7443	User, Administrator role	—	Host cluster	Pool	Default path policy `alua`
Pure Storage FlashArray	`pure`	443	API token of a Storage Admin user (no username)	—	Host group	`default`	Always thin; deleted volumes recoverable until eradicated
INFINIDAT InfiniBox	`infinidat`	443	User, Pool Admin or Admin	—	Cluster	Pool	Thin or thick
Dell PowerStore	`dell-powerstore`	443	User, Storage Administrator	—	Host group	`default`	Always thin
Dell PowerMax / VMAX	`dell-powermax`	8443 (Unisphere)	Unisphere user, StorageAdmin	`--system-id` (Symmetrix ID), `--portgroup` (required); `--srp`, `--slo`	Host group, storage group `<cluster>_SG`, masking view `<cluster>_MV`	The SRP of `<cluster>_SG`	Create a port group of the zoned array FC ports first
NetApp ONTAP (AFF / ASA / FAS)	`netapp-ontap`	443	Cluster or SVM admin with REST access	`--svm` (required)	igroup (FCP, iSCSI or mixed)	Aggregate	One FlexVol (`/vol/<name>/lun0`) per datastore; names use `_` instead of `-`
NetApp E-Series	`netapp-eseries`	8443	Storage admin	`--system-id` (default `1`)	Host group	Pool / DDP	Thick volumes only; names up to 30 characters
Hitachi VSP	`hitachi-vsp`	443 (embedded) or your Configuration Manager port	Storage Administrator (Provisioning)	`--target-ports` (required, e.g. `CL1-A,CL2-A`); `--system-id`	One host group per target port (host mode LINUX/IRIX)	DP pool	LU path on every target port; WWID known after mapping

Array	`--type`	Port	Credentials	Extra settings	Cluster object pvesan creates	Pool	Notes
Fujitsu ETERNUS DX S5 / AF S3	`fujitsu-eternus`	5665 (enable the RESTful API and open the port in the firewall settings)	User with the Admin or StorageAdmin role	—	One FC host per HBA port (`<node>_<n>`)	Thin provisioning pool (TPP)	Firmware V11L40 or later; host affinity enabled on the CA ports; the cluster's hosts must not share a LUN group with other hosts
HPE Primera / Alletra 9000 / 3PAR StoreServ	`hpe-3par`	443 (Primera, Alletra 9000); 8080 on 3PAR StoreServ (3PAR OS 3.x)	User with the edit role; WSAPI started (`startwsapi`)	—	Host set (hosts with persona 2, Generic-ALUA); volumes exported to the host set (`set:<name>`) with one LUN	CPG	Thin by default; thick (full) only on 3PAR StoreServ; volume and host names up to 31 characters, the host set up to 27
HPE Alletra 5000 / 6000 (Nimble)	`hpe-nimble`	5392 (NimOS REST API)	NimOS user with the Power User or Administrator role	—	One FC initiator group holding every node's WWPNs	Pool	Thin (reserve 0) or thick (reserve 100%); WWPNs already in another initiator group are refused; names use letters, digits, `-` and `.`
Dell Unity XT	`dell-unity`	443	Unisphere user with the Storage Administrator role	—	One host per node with its FC initiators; every LUN's host access lists all of them	Pool	Thin or thick; zone the nodes first so the array sees their initiators log in (otherwise pvesan takes each port's WWNN from the node). Copies of snapshots are thin clones (thin LUNs, single-LUN datastores)

Array snapshots (P13 to P15) work with `pure`, `infinidat`, `dell-powerstore`, `netapp-ontap`, `ibm-sv`, `hpe-3par`, `hpe-nimble` and `dell-unity`. `dell-powermax`, `netapp-eseries`, `hitachi-vsp` and `fujitsu-eternus` report them as not supported yet.

First use with a new array type: before your first production datastore, provision and delete a 10 GiB test datastore. Every driver's calls were checked against the vendor's own SDK or tooling, but real-array results are still being collected. If a step fails, the task log shows the array's own error code; please report it.

P13. Array snapshots of a datastore

Array snapshots are taken by the array in seconds and cost no host I/O. They work for array-linked datastores on **Pure FlashArray**, **INFINIDAT InfiniBox**, **Dell PowerStore**, **NetApp ONTAP**, **IBM Spectrum Virtualize**, **HPE Primera / Alletra 9000 / 3PAR**, **HPE Alletra 5000/6000 (Nimble)** and **Dell Unity XT**; the other drivers report "not supported yet". A datastore on several array volumes is snapshotted at one point in time: Pure uses a protection group `pvesan-<first volume>`, InfiniBox a group snapshot, PowerStore a write-order consistent volume group `pvesan-<first volume>`, IBM a FlashCopy consistency group, HPE 3PAR a snapshot of the volume set `pvesan-<first volume>`, Nimble a volume collection `pvesan-<first volume>` and its snapshot collections (a volume can be in one volume collection only, so a volume already in another one is refused), Unity a consistency group `pvesan-`

`<first volume>` (the LUNs stay in it). On ONTAP only single-LUN datastores can be snapshotted yet (one FlexVol per LUN; several would need a consistency group). On Unity a snapshot of a multi-LUN datastore can be rolled back but not mounted as a copy: Unity cannot thin-clone a member of a consistency group snapshot.

Snapshots are **crash-consistent**: running VMs are not paused, so their disks look as if the power had been cut at that moment. For application consistency, shut the VM down or quiesce the application first.

GUI: **SAN** → **Snapshots**, pick the datastore, then **Take snapshot**, **Delete**, **Mount as copy**, **Roll back datastore** or **Schedule**.

```
pvesan snapshot create gold01 --name before-upgrade   # default name pvesan-<date>-<time>
pvesan snapshot list gold01
pvesan snapshot delete gold01 before-upgrade
```

Names use letters, digits and `-` and start with a letter. On the array the snapshot is named `<volume>.<name>` (Pure: `<volume>.<name>` or `<protection group>.<name>.<volume>`; ONTAP: a snapshot `<name>` of the LUN's FlexVol; IBM: a thin FlashCopy target volume `pvs_<volume id>_<name>`; HPE: a virtual copy `<volume>.<time in hex>` that carries the pvesan name in its comment, because 3PAR names are limited to 31 characters). On Pure a deleted snapshot stays recoverable on the array until it is eradicated (24 h by default).

Schedules. `pvesan snapshot schedule gold01 --snapshot-interval 4 --snapshot-keep 12` takes a snapshot every 4 hours and keeps the newest 12 scheduled ones. Scheduled snapshots are named `auto-<date>-<time>`; manual snapshots and mounted snapshots are never removed by the schedule. `pvesan-snapshot.timer` runs on every node every 15 minutes, and a claim under the cluster-wide pvesan lock makes sure exactly one node takes each snapshot. `pvesan snapshot schedule-show gold01` shows the last run and its error, if any; remove a schedule with `--delete snapshot-interval`.

P14. Mount a snapshot and restore a VM

A snapshot is never mounted over the live datastore. **Mount as copy** (`pvesan snapshot mount gold01 before-upgrade`) in one task:

1. Clones the snapshot to a new array volume (`<copy>`), maps it to the cluster and waits until every node sees it.
2. Imports its volume group under a new name with `vgimportclone`, which gives the PVs and the VG new UUIDs, so it can't collide with the live datastore.
3. Makes every volume in it read-only and registers it as datastore `<datastore>-snap-<date>-<time>` (or `--copy <id>`).

Only the node that runs the task sees the clone until `vgimportclone` has given it new PV and VG UUIDs; the other nodes are rescanned after that, so they never see two PVs with the same identity. All nodes must be online.

Restore a VM from the copy (**Datastores** → **Restore VM**, or the CLI):

```
pvesan copy restore-vm gold01-snap-20261029-0800 101                    # over the live disks
pvesan copy restore-vm gold01-snap-20261029-0800 101 --target-vmid 9101   # into a new VM
```

- **Over the live disks:** the VM must be stopped and must not have Proxmox snapshots. Each disk of the VM on the datastore that exists in the snapshot is overwritten with its snapshot content. Disks added after the

snapshot are left alone and reported.
- **Into a new VM ID:** new disks are allocated (on the original datastore, or `--target-storage`), the snapshot content is copied in, and a config is written on the node that runs the task. Disks on other storages and unrestored disks are left out, MAC addresses, the SMBIOS UUID and the VM generation ID are new, and every network link starts down (`link_down=1`), so the copy can't clash with the original. Check the config, then bring the links up.

The guest's configuration is taken from its current config, not from the snapshot. Containers are restored the same way (`pvesan copy restore-vm <copy> <ctid>`): their root disk and mount points on the datastore are copied, and a new container ID gets a new MAC address and its links down.

When you are done, **Remove copy** (`pvesan copy remove gold01-snap-20261029-0800`) deletes the copy's storage, volume group and clone volume; the snapshot stays. A snapshot that is mounted can't be deleted, and a datastore with mounted copies can't be deleted or rolled back.

P15. Roll back a whole datastore

`pvesan snapshot rollback gold01 before-upgrade --confirm gold01` (GUI: **Roll back datastore**, type the ID) restores every volume of the datastore in place from the snapshot. **Everything written to the datastore since the snapshot is lost**, for every VM on it.

pvesan refuses unless every guest using the datastore is stopped, every node of the datastore is online and no copy of it is mounted. It then disables the datastore, deactivates its volume group on all nodes, restores the array volumes (PowerStore keeps a backup snapshot of the state before the restore), makes every node re-read the LUNs and enables the datastore again. Guests whose config refers to disks that did not exist at the snapshot are listed as warnings; remove those disk entries before starting them.

P16. Automatic space reclaim and discard on delete

- **Weekly reclaim:** `pvesan snapshot schedule gold01 --auto-reclaim 1` (GUI: **Snapshots** → **Schedule** → **Weekly reclaim**) runs P8 once a week. `pvesan-reclaim.timer` fires on every node on Sunday night; one node that has the datastore claims each run. It works on any datastore whose LUNs support UNMAP, with or without an array driver.
- **Discard on delete:** `pvesm set gold01 --fc-discard 1` sends UNMAP for a VM disk's blocks just before the disk is removed, so a thin array frees the space at once. On Proxmox VE with libpve-storage-perl 9.1 or later it turns on the LVM plugin's own `on-volume-remove=discard=1`, which discards in a background task after the disk is removed from the volume group, so large disks are fine. On older versions the discard runs inside Proxmox's storage lock, which times out after 60 s; there, keep it off for very large disks on a slow array and rely on the weekly reclaim. It is ignored on datastores with `saferemove`, which already wipe removed disks.

P17. Per-disk array volumes (`fcsanvol`, experimental)

> **Experimental.** `fcsanvol` works with Pure FlashArray only, takes raw VM disks only and has not yet run against a real array. Its options and behaviour may change in a later release. Use it with test VMs, not production guests.

Storage type `fcsanvol` gives every VM disk its own array volume, the Proxmox equivalent of vVols: disks are thin on the array, and Proxmox's own snapshot, rollback, template, clone and resize actions become array operations that take seconds. It needs an array driver with per-disk volume support; so far that is **Pure FlashArray** (`pure`). Shared-LVM datastores stay as they are and work on every array.

```
pvesm add fcsanvol fa-vols --fc-array fa1 --fc-pool default --content images
```

GUI: **SAN** → **Arrays**, select the array, **Per-disk storage**.

Proxmox action	What happens on the array
Create a disk	A new volume `<storage>-vm-<id>-disk-<n>`, connected to the cluster's host group; the LUN number is recorded
Start or migrate the VM	The node scans only that LUN number on its adapters (`- - <lun>`), not the whole fabric, and waits for the multipath map `/dev/mapper/<storage>-vm-<id>-disk-<n>`
Snapshot / rollback / delete snapshot	Volume snapshot / overwrite from the snapshot / destroy the snapshot. Snapshot names must use letters, digits and `-` (no `_`)
Convert to template	The volume is renamed to `...-base-<id>-disk-<n>`
Linked clone, clone from a snapshot	An array copy: seconds, no space until written
Resize	Volume expand, then the node running the VM re-reads the LUN
Delete the disk	Paths removed on every node, volume disconnected and destroyed (recoverable on Pure until eradicated)

Limits. Every disk is a LUN mapped to every node. `fc-max-luns` (default 255) stops new disks before the array's or HBA's LUN limit; raise it only if your array and HBAs allow more LUNs per host. Raw disks only: no containers, no cloud-init drive and no qcow2. When a VM stops or migrates away, its disks' maps and paths are removed from that node (the array mapping stays, and the next start scans the LUN back in); `--fc-keep-luns 1` keeps them instead. A map still open on the node is kept and the task log says so. Disk moves between `fcsanvol` and other storages use Proxmox's normal host copy. This storage type has not run on a real array yet; try it with a test VM first.

P18. Replication and disaster recovery (experimental)

> **Experimental.** Replication setup works with Pure FlashArray only and has not yet run against real arrays. The DR-site import (`pvesan dr import`) works with LUNs from any array. Try the whole cycle with a test datastore and a test failover before you rely on it.

Two Proxmox VE clusters, each with its own array: the primary cluster's array replicates the datastore to the DR array, and the DR cluster brings it up from the replica. Add each array to the cluster at its own site (P12) and connect the two arrays for asynchronous replication on the arrays themselves (on Pure: an array connection of type `async-replication`).

At the primary site

```
pvesan dr partners fa-prod                      # arrays fa-prod can replicate to
pvesan dr enable gold fa-dr --interval 60 --retain 24
pvesan dr set-plan gold 101:60,102,103          # start order at the DR site, 60 s after 101
pvesan dr status gold                           # newest complete replica, its age, transfer in
progress
pvesan dr replicate gold --wait 1               # refresh the guest configs and send a replica now
```

`enable` puts all volumes of the datastore in one protection group, so they replicate at one point in time (crash-consistent), and writes the configs of its guests onto the datastore (LV `pvesan-dr-meta`). `pvesan-dr.timer` refreshes them every 15 minutes, so a guest created or changed since then reaches the DR site with the next replica after that. The recovery plan is carried the same way. Guests outside the plan are created at the DR site but not started.

Test failover (at the DR site; production is not touched)

```
pvesan dr replicas fa-dr                        # replicas, newest complete one, its age
pvesan dr test fa-dr fa-prod:pvesan-gold --start 1
#   ... check the guests, the applications, the timings ...
pvesan dr cleanup gold-drtest
```

The test copies the newest complete replica (or `--snapshot <name>`) to new volumes, maps them to this cluster, imports the volume group as `san_gold-drtest` and registers datastore `gold-drtest`. Its guests get new MAC addresses, SMBIOS UUIDs and `-drtest` names, and their network links start down: connect them to an isolated bridge before you set `link_down=0`. If the VM IDs are taken at this site, add `--vmid-offset 10000`: the disks are renamed to match. `cleanup` stops and removes the test guests, the datastore and the copied volumes.

Planned failover

1. At the primary site: `pvesan dr prepare-failover gold --confirm gold --shutdown 1`. This shuts the guests down in reverse plan order, writes their configs, unmounts the datastore on every node and sends a final replica `dr-final-<date>-<time>`, then prints the command for the DR site.
2. At the DR site: `pvesan dr failover fa-dr fa-prod:pvesan-gold --snapshot dr-final-<date>-<time> --confirm fa-prod:pvesan-gold --start 1`.

Unplanned failover (the primary site is gone): step 2 alone, from the newest complete replica. Make sure the primary site's guests cannot come back while the DR copies run (fence or power off the primary cluster).

A failover keeps the guests' identities (MACs, UUIDs, VM IDs unless you set an offset) and keeps the storage ID `gold`, so the configs need no changes. The new datastore is linked to the DR array and carries the recovery plan.

Failing back. Replicate the DR site's datastore back with `pvesan dr enable gold fa-prod` at the DR site, wait for a complete replica, then run the planned failover in the other direction. Delete the old datastore at the primary site first if its storage ID would clash (`pvesan datastore delete gold --delete-volume 0` keeps the array volume).

LUNs promoted by hand (any array). If the array's own tools promoted or copied the replica volumes and they are mapped to the DR cluster, bring them up with `pvesan dr import gold --wwids <wwid>[,<wwid>...]` (add `--test 1` for an isolated test copy). `cleanup` of such a test copy releases the LUNs and leaves their data alone.

Limits. Replicas are crash-consistent. Only the current state of a guest is brought up, not its Proxmox snapshots. HA resources, firewall settings outside the guest config, backup jobs and pool membership stay at the primary site and need to be set up at the DR site. Disks on other storages are kept only if that storage exists at the DR site. A CD-ROM image that doesn't exist there is emptied.

P19. iSCSI arrays

pvesan provisions on iSCSI arrays the same way as on FC: it registers each node's iSCSI initiator name on the array instead of (or as well as) its WWPNs, and logs every node in to the array's target portals. Every driver except `fujitsu-eternus` supports it; that one says "iSCSI host registration is not supported yet".

Before you start

1. Give every node an interface on the array's iSCSI network (ideally two, on separate subnets or VLANs, for multipath) and check that each node can reach every portal (`ping`, or `nc -zv <portal> 3260`).
2. `open-iscsi` is installed on Proxmox VE by default. Each node's initiator name is in `/etc/iscsi/initiatorname.iscsi`, and `pvesan iscsi status` shows it with the node's sessions. Initiator names must be unique: if nodes were cloned from one image, give each its own (`echo "InitiatorName=$(iscsi-iname)" > /etc/iscsi/initiatorname.iscsi`, then `systemctl restart iscsid`) before registering.
3. If the array needs CHAP, set it in `/etc/iscsi/iscsid.conf` on every node first. pvesan doesn't manage CHAP secrets.
4. Array specifics:
    - **NetApp ONTAP:** the SVM needs the iSCSI service and data LIFs with the `data_iscsi` service. The igroup is created with protocol `iscsi` (or `mixed` with `--protocol both`). An existing `fcp` igroup can't take iSCSI initiators: ONTAP doesn't allow changing an igroup's protocol, so use another `--hostcluster` name.
    - **Hitachi VSP:** list the iSCSI target ports in `--target-ports`. pvesan creates the host group (an iSCSI target) on each port and registers the initiator names in it (`host-iscsis`). With `--protocol both`, FC ports get WWPNs and iSCSI ports get initiator names.
    - **Dell PowerMax:** `--portgroup` must be a port group of iSCSI target ports. pvesan reads their IP addresses and target IQNs from it.
    - **IBM Spectrum Virtualize:** portals are the nodes' host-attach Ethernet ports (`lsportip`, or `lsip` portset `portset0` on 8.4.2 and later).

Register and log in

```
pvesan array add fa01 --type pure --address 192.0.2.20 --fingerprint 'sha256$...' --protocol iscsi
# or, for an array that is already added:
pvesan array set fa01 --protocol iscsi        # 'both' for FC and iSCSI hosts
pvesan array register-cluster fa01
pvesan array status fa01
```

`register-cluster` creates or updates the host objects with each node's initiator name and puts them in the host cluster. Then every node:

1. runs `iscsiadm -m discovery -t sendtargets -p <portal>` for each portal the array reports,
2. sets `node.startup=automatic` on every discovered node record,

3. logs in (`iscsiadm -m node -T <iqn> -p <portal> --login`) wherever it has no session yet.

Each step is idempotent: run it again at any time. In the GUI, **Arrays → Registration** has **Register cluster on array** and **iSCSI login**.

If the driver can't list the portals, or you want other ones, give them yourself:

```
pvesan array iscsi-login fa01 --portals 192.0.2.21,192.0.2.22
pvesan iscsi login 192.0.2.21               # this node only
pvesan iscsi status                         # initiator, sessions, recorded portals
```

Then provision as in P12: `pvesan provision fa01 gold01 --pool default --size 2048`. Check that every LUN has a path through each portal (`pvesan devices`, `multipath -ll`).

At boot, and for new nodes

Every portal a node logs in to is recorded in `/etc/pve/pvesan/config.json` (`iscsi.portals`). At boot, `pvesan-sync.service` runs `pvesan iscsi restore` before `pvesan sync`, which logs in to every recorded portal the node has no session to. That also covers nodes that join the cluster later. The monitor registers a new node's initiator name on arrays with auto-register on, and logs it in to their portals. open-iscsi's own `node.startup=automatic` also restores the sessions.

`pvesan array remove` drops the array's recorded portals. The nodes stay logged in until they reboot or you log them out (`iscsiadm -m node -T <iqn> -p <portal> --logout`, then `-o delete`).

Troubleshooting

Symptom	Fix
`iSCSI host registration is not supported yet`	The driver has no verified iSCSI calls. Register the hosts on the array by hand and use `pvesan iscsi login`, or use FC.
`array ... reports no iSCSI target portals`	Use `pvesan array iscsi-login <array> --portals ...`
`no iSCSI session to portal ... could be established`	Check the network path, CHAP, and that the host object holds this node's initiator name (`pvesan array hosts <array>`).
LUN not visible after login	Check the host is in the host cluster (`pvesan array status`), then `pvesan rescan`.

Troubleshooting and recovery

Start from the task log. Every cluster-wide step prints `[node] FAILED: <reason>`, and that reason is the stderr of the LVM or multipath command, or the array's error message.

Symptom	Likely cause	Fix
`LUN ... not visible` on create or extend	The node isn't zoned, or its WWPN isn't in the array host group	Fix zoning or masking, run P1, try again

Symptom	Likely cause	Fix
`nodes disagree about the size` on extend	Some nodes still see the old capacity, or the LUN ID differs per node	Run P1 on all nodes; check every node maps the same LUN
Datastore `inactive`, `backing LUN(s) not present`	The LUN was unmapped or the paths are down on that node	`pvesan paths`, `pvesan adapters`; fix the fabric, then `pvesan rescan`
Device flag `not multipathed` on a datastore LUN	The node was offline when the datastore was created	`pvesan sync` on that node (also runs at boot)
`Found duplicate PV` warnings	LVM sees the raw `sdX` paths	Make sure `multipath_component_detection = 1`, then `pvesan sync`
Paths `faulty` or `offline` after an array change	The LUN was unmapped before it was detached	`pvesan device cleanup`; if the map is still open, stop what uses it first
Device flag `resize pending`	The LUN grew but the map didn't	`pvesan datastore extend <id>` or `pvesan device refresh <wwid>`
GUI panel missing after a Proxmox VE update	The integration was overwritten and the trigger failed	`/usr/share/pve-fcsan/pvesan-integrate apply`, then Ctrl+F5
`'<name>'-locked command timed out`	A cluster lock was held for more than 60 s	Retry; report it if it happens again
Array step fails	Credentials, role, certificate or array state	Read the array error code in the task log; check the user's role and `pvesan array status <id>`

Useful commands

```
multipath -ll
multipathd show paths format "%d %t %T %o %p %m"
pvs; vgs; lvs
cat /etc/multipath/conf.d/pvesan.conf
cat /etc/pve/pvesan/config.json
journalctl -u pvesan-sync -u multipathd -b
```

Mount a snapshot or replica LUN by hand (manual resignature). With a driver that supports snapshots, use P14 instead. A copy of a datastore LUN carries the same VG name and PV UUIDs as the original, so don't present it to nodes that see the original until it has a new identity. On one node:

1. Present the copy, then run `pvesan rescan` and `pvesan device add <copy-wwid> --alias ds1-snap`.
2. Run `vgimportclone --basevgname san_ds1_snap /dev/mapper/ds1-snap` to give it new UUIDs and a new VG name.
3. `pvesm add lvm ds1-snap --vgname san_ds1_snap --shared 1`, or keep it local to restore single disks.

Lost `/etc/pve/pvesan/config.json`. Datastores keep working, because LVM metadata lives on the LUNs. Aliases and policies fall back to multipath defaults, and arrays must be added again. Re-run `pvesan device set` for each LUN, or restore the file from a backup of `/etc/pve`.

Monitoring and health

`pvesan-monitor.timer` runs `pvesan monitor-run` on every node every 5 minutes. Each node checks its own paths; one node (it takes a lease in `/etc/pve/pvesan/monitor.json`) reads every array, caches what it read in the same file and registers new nodes. Problems are sent through Proxmox VE's notification system (**Datacenter** → **Notifications**: mail, webhook, Gotify, ...) when they appear and again when they are resolved, never every 5 minutes. To route or silence them, add a matcher on the field `type` = `pvesan`. They are also written to the journal (`journalctl -u pvesan-monitor`).

H1. Array view of a datastore

Datastores → **Array view** (`pvesan datastore array-view gold01`) shows, for an array-linked datastore:

- the pool(s) of its volume(s): capacity, free and used percentage;
- each volume: provisioned size, written data and data reduction;
- the array's open alerts and overall state.

The view comes from the cache (at most 15 minutes old); **Read array now** (`--refresh 1`) reads the array at once. What each driver reports:

Driver	Pool capacity / free	Provisioned	Written	Data reduction	Array health
`pure`	yes	yes	yes	yes (ratio)	yes (open alerts)
`infinidat`	yes	yes	yes	capacity saved	not yet
`dell-powerstore`	not reported by the API pvesan uses	yes	yes (PowerStoreOS 3+)	not yet	not yet
other drivers	as their pool list	yes	not yet	not yet	not yet

A pool fuller than the threshold (85 % by default) raises an alert: `pvesan monitor set --pool-threshold 90`, or **Health** → **Pool alert threshold**.

H2. Performance per LUN

Performance (`pvesan iostats`) shows, for every multipath LUN on the node you are looking at: read and write IOPS, throughput, average latency, how busy the LUN was and the I/O in flight. The numbers come from the block statistics of each device-mapper multipath map, measured over 2 seconds (`--interval 1..10`); the tab refreshes every 10 seconds while it is open. They are the node's view; the array's own statistics are not read yet.

H3. Which VMs are on a datastore, and which LUN a VM is on

- **Datastores** → **VMs** (`pvesan datastore guests gold01`) lists every VM and container with disks on the datastore, its node, state and disks, from the guest configs of all nodes.
- `pvesan guest 101` lists each disk of a guest with its datastore, volume group, LUN(s) and array volume(s).

H4. Path health and alerts

Every node checks that each LUN of each enabled datastore it should see is present and has all its paths. It remembers the highest path count it has seen per LUN, so a path that disappeared entirely (not just failed) is still reported. Problems:

Problem	Meaning
`LUN ... is not visible`	The datastore's LUN is missing on this node
`has no active path`	All paths are down; I/O is queued or fails, depending on `no_path_retry`
`has N of M paths active`	A fabric, HBA port or array port is down, or a path was removed
`pool ... is N% full`	An array pool passed the threshold
`array ... reports warning/critical`	The array has open alerts (Pure)
`array ... could not be read`	An array API call failed (credentials, certificate, network)
`registering new nodes ... failed`	See H5

Health (`pvesan path-health`) shows the current problems of a node. After an intended change that removes paths (rezoning, fewer HBA ports), click **Forget path counts** (`pvesan path-health-reset`) on each node so the lower count becomes normal.

H5. Nodes joining and leaving

- **Join:** when the cluster's membership changes, the monitor registers the FC ports of every node that has no host on the array yet, on every array with `auto-register` (the default), exactly as **Register cluster** would. A node whose ports already belong to a host is left alone. Turn it off per array with `pvesan array set fs01 --auto-register 0`.
- **Leave:** after `pvecm delnode`, **Arrays → Hosts** (`pvesan array hosts fs01`) marks hosts in the cluster's host cluster that belong to no cluster member as **Removed node**. **Remove host** (`pvesan array remove-host fs01 <host>`) takes such a host out of the host cluster and deletes it on the array. pvesan refuses while any node is offline (its host could not be told apart) and for any host that is not marked. Supported on `pure`, `infinidat` and `dell-powerstore`; on other arrays remove the host in the array's GUI.

H6. Zoning hints

Arrays → Zoning (`pvesan array zoning fs01`) compares the array's FC target ports with the ports each node's adapters are logged in to, and names what is missing: an adapter logged in to none of the array's ports (zone missing on that fabric), or a node that reaches some array ports but not others. The array's ports come from the array where the driver can read them (`pure`); otherwise from the target ports that any node uses to reach the array's LUNs. pvesan does not change the switches.

H7. All paths down: no_path_retry

`no_path_retry` decides what multipath does with I/O when every path to a LUN is gone: keep it queued for that many path checks (5 s each by default), queue it forever (`queue`), or fail it at once (`fail`). Queueing forever means a VM hangs until a path returns, and on an HA cluster the node does not fence itself. pvesan leaves multipath-tools'

built-in value for each array unless that value is "queue forever"; for those arrays it sets **12** (about one minute) on LUNs it provisions:

Array	multipath-tools built-in `no_path_retry`	Set by pvesan on provisioned LUNs
IBM FlashSystem / Spectrum Virtualize (2145)	`queue`	12
NetApp ONTAP (`NETAPP LUN`)	`queue`	12
INFINIDAT InfiniBox	`queue`	12
Pure FlashArray	not set (no queueing)	—
Dell PowerStore	3	—
Dell PowerMax / VMAX	6	—
NetApp E-Series	30	—
Hitachi VSP (`OPEN-`)	10	—
Fujitsu ETERNUS DX	10	—
Dell Unity XT (`DGC`)	60	—
HPE Primera / Alletra 9000 / 3PAR (`3PARdata VV`)	18	—

The built-in values are from multipath-tools' hardware table and change between versions; `multipath -t` shows the ones in force on a node. Set or change the value per LUN in **Devices** → **Properties** → **No path retry** or with `pvesan device set <wwid> --no-path-retry 12` (`--delete no-path-retry` returns to the built-in value); it applies on every node.

CLI and API reference

Each `pvesan` command calls the API method in the same row, so `pvesan help <command>` and the API viewer show the same parameters. List commands accept `--output-format json|yaml|text`. Cluster tasks exit non-zero when they fail.

CLI	REST (under `/nodes/{node}/san`)	Scope	Key options
`pvesan adapters`	`GET adapters`	node	
`pvesan remote-ports`	`GET remote-ports`	node	
`pvesan devices`	`GET devices`	node	`--all`, `--unused`
`pvesan paths`	`GET paths`	node	
`pvesan policies`	`GET policies`	—	
`pvesan rescan`	`POST rescan`	node	`--lip`, `--resize`
`pvesan rescan-all`	`POST rescan-all`	cluster task	`--lip`
`pvesan sync`	`POST sync`	node	

CLI	REST (under `/nodes/{node}/san`)	Scope	Key options
`pvesan iscsi status`	`GET iscsi`	node	
`pvesan iscsi login <portal>`	`POST iscsi-login`	node	`--target`, `--array`
`pvesan iscsi restore`	`POST iscsi-restore`	node (boot)	
`pvesan iostats`	`GET iostats`	node	`--interval`
`pvesan path-health`	`GET path-health`	node	
`pvesan path-health-reset`	`POST path-health-reset`	node	
`pvesan guest <vmid>`	`GET guests/{vmid}`	cluster view	
`pvesan monitor show` / `set`	`GET` / `PUT monitor-settings`	cluster	`--pool-threshold`
`pvesan monitor-run`	`POST monitor-run`	timer	
`pvesan device add <wwid>`	`POST devices/{wwid}/claim`	cluster task	`--alias`
`pvesan device set <wwid>`	`PUT devices/{wwid}`	cluster task	`--alias`, `--policy`, `--no-path-retry`, `--delete`
`pvesan device detach <wwid>`	`POST devices/{wwid}/detach`	cluster task	
`pvesan device attach <wwid>`	`POST devices/{wwid}/attach`	cluster task	
`pvesan device refresh <wwid>`	`POST devices/{wwid}/refresh`	node	
`pvesan device cleanup`	`POST cleanup-paths`	node	
`pvesan datastore list`	`GET datastores`	cluster view	
`pvesan datastore create <id>`	`POST datastores`	cluster task	`--devices`, `--label`, `--vgname`, `--content`, `--nodes`, `--saferemove`, `--force`
`pvesan datastore extend <id>`	`POST datastores/{id}/extend`	cluster task	`--add-devices`, `--devices`, `--force`
`pvesan datastore array-grow <id>`	`POST datastores/{id}/array-grow`	cluster task	`--size <GiB>`

CLI	REST (under `/nodes/{node}/san`)	Scope	Key options
`pvesan datastore array-view <id>`	`GET datastores/{id}/array-view`	cluster view	`--refresh`
`pvesan datastore guests <id>`	`GET datastores/{id}/guests`	cluster view	
`pvesan datastore rename <id>`	`POST datastores/{id}/rename`	cluster task	`--label`, `--new-id`, `--rename-vg`
`pvesan datastore unmount <id>`	`POST datastores/{id}/unmount`	cluster task	`--nodes`
`pvesan datastore mount <id>`	`POST datastores/{id}/mount`	cluster task	`--nodes`
`pvesan datastore reclaim <id>`	`POST datastores/{id}/reclaim`	cluster task	`--reserve <GiB>`
`pvesan datastore delete <id>`	`DELETE datastores/{id}`	cluster task	`--destroy-data`, `--detach`, `--delete-volume`
`pvesan array list`	`GET arrays`	cluster view	
`pvesan array probe <address>`	`POST array-probe`	node	`--port`, `--type`
`pvesan array add <id>`	`POST arrays`	cluster	`--type`, `--address`, `--port`, `--username`, `--fingerprint`, `--hostcluster`, `--auto-register`, `--protocol` (`fc`, `iscsi`, `both`), vendor settings
`pvesan array set <id>`	`PUT arrays/{id}`	cluster	as add, `--delete`
`pvesan array remove <id>`	`DELETE arrays/{id}`	cluster	
`pvesan array status <id>`	`GET arrays/{id}/status`	cluster view	
`pvesan array pools <id>`	`GET arrays/{id}/pools`	—	
`pvesan array register-cluster <id>`	`POST arrays/{id}/register-cluster`	cluster task	
`pvesan array iscsi-login <id>`	`POST arrays/{id}/iscsi-login`	cluster task	`--portals`

CLI	REST (under /nodes/{node}/san)	Scope	Key options
`pvesan array link <id> <storage>`	`POST arrays/{id}/link`	cluster	
`pvesan array zoning <id>`	`GET arrays/{id}/zoning`	cluster view	
`pvesan array hosts <id>`	`GET arrays/{id}/hosts`	cluster view	
`pvesan array remove-host <id> <host>`	`POST arrays/{id}/remove-host`	cluster task	
`pvesan provision <array> <storage>`	`POST arrays/{id}/provision`	cluster task	`--pool`, `--size`, `--thin`, `--volume`, `--label`, `--content`, `--policy`
`pvesan snapshot list <id>`	`GET datastores/{id}/snapshots`	cluster view	
`pvesan snapshot create <id>`	`POST datastores/{id}/snapshots`	cluster task	`--name`
`pvesan snapshot delete <id> <name>`	`DELETE datastores/{id}/snapshots/{name}`	cluster task	
`pvesan snapshot mount <id> <name>`	`POST datastores/{id}/snapshots/{name}/mount`	cluster task	`--copy`
`pvesan snapshot rollback <id> <name>`	`POST datastores/{id}/snapshots/{name}/rollback`	cluster task	`--confirm <id>`
`pvesan snapshot schedule <id>`	`PUT datastores/{id}/schedule`	cluster	`--snapshot-interval <h>`, `--snapshot-keep`, `--auto-reclaim`, `--delete`
`pvesan snapshot schedule-show <id>`	`GET datastores/{id}/schedule`	cluster view	
`pvesan copy restore-vm <copy> <vmid>`	`POST datastores/{copy}/restore-vm`	cluster task	`--target-vmid`, `--target-storage`
`pvesan copy remove <copy>`	`POST datastores/{copy}/remove-copy`	cluster task	
`pvesan dr partners <array>`	`GET arrays/{id}/replication-partners`	—	

CLI	REST (under `/nodes/{node}/san`)	Scope	Key options
`pvesan dr enable <id> <partner>`	`POST datastores/{id}/replication`	cluster task	`--interval <min>`, `--retain <h>`
`pvesan dr disable <id>`	`DELETE datastores/{id}/replication`	cluster task	
`pvesan dr status <id>`	`GET datastores/{id}/replication`	cluster view	
`pvesan dr replicate <id>`	`POST datastores/{id}/replicate`	cluster task	`--name`, `--wait`
`pvesan dr write-meta <id>`	`POST datastores/{id}/dr-meta`	cluster task	
`pvesan dr plan <id>` / `set-plan <id> <order>`	`GET` / `PUT datastores/{id}/recovery-plan`	cluster	
`pvesan dr prepare-failover <id>`	`POST datastores/{id}/prepare-failover`	cluster task	`--confirm <id>`, `--shutdown`
`pvesan dr replicas <array>`	`GET arrays/{id}/replicas`	cluster view	
`pvesan dr snapshots <array> <replica>`	`GET arrays/{id}/replicas/{replica}/snapshots`	cluster view	
`pvesan dr test <array> <replica>`	`POST arrays/{id}/replicas/{replica}/test`	cluster task	`--snapshot`, `--storage`, `--vmid-offset`, `--start`
`pvesan dr failover <array> <replica>`	`POST arrays/{id}/replicas/{replica}/failover`	cluster task	as test, `--confirm <replica>`
`pvesan dr import <id> <wwids>`	`POST dr-import`	cluster task	`--test`, `--vmid-offset`, `--start`
`pvesan dr run-plan <id>`	`POST datastores/{id}/run-plan`	cluster task	
`pvesan dr cleanup <id>`	`POST datastores/{id}/dr-cleanup`	cluster task	
`pvesan dr-run`	`POST dr-run`	timer	
`pvesan snapshot-run`	`POST snapshot-run`	timer	

CLI	REST (under `/nodes/{node}/san`)	Scope	Key options
`pvesan reclaim-run`	`POST reclaim-run`	timer	

There are also two internal steps, `POST devices/{wwid}/detach-local` and `POST vg-deactivate`. Cluster tasks call them on each node; don't call them by hand.

Storage options added by the `fcsan` type. All LVM options (`vgname`, `shared`, `saferemove`, `nodes`, `content`, `snapshot-as-volume-chain` on Proxmox VE 9, ...) also apply.

Option	Default	Meaning
`fc-wwids`	set at create	WWIDs of the backing LUNs, comma separated
`fc-autorescan`	`1`	Rescan the SAN on activation when a backing LUN is missing
`fc-discard`	`0`	UNMAP a volume's blocks when it is removed (P16): the LVM plugin's background discard where available, otherwise inline and ignored with `saferemove`
`fc-keep-luns` (`fcsanvol`)	`0`	Keep a disk's LUN on a node after its VM stops or leaves (P17)

Storage options of the `fcsanvol` type (P17):

Option	Default	Meaning
`fc-array`	required, fixed	pvesan array the disk volumes are created on
`fc-pool`	required	Array pool for new volumes (`default` on Pure)
`fc-max-luns`	`255`	Most disk volumes on this storage

Example entry:

```
fcsan: ds1
        vgname san_ds1
        content images,rootdir
        shared 1
        fc-wwids 3600a098038303053453f463045727a41
```